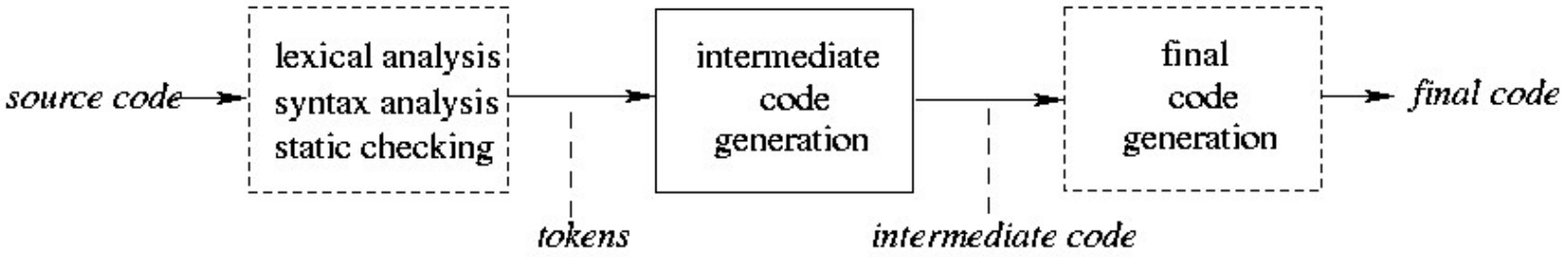
Intermediate Code Generation

Part - 01

ICG

D. VIJAYAKUMAR,
AP(SG) / CSE

Overview



- Intermediate representations span the gap between the source and target languages:
  - closer to target language;
  - (more or less) machine independent;
  - allows many optimizations to be done in a machine-independent way.
- Implementable via syntax directed translation, so can be folded into the parsing process.

Issues in new IR Design

- Source language and target language
- Porting cost or reuse of existing design
- There is no standard Intermediate Representation.
- IR is a step in expressing a source program so that machine understands it
- As the translation takes place, IR is repeatedly analyzed and transformed
- Compiler users want analysis and translation to be fast and correct
- Compiler writers want optimizations to be simple to write, easy to understand and easy to extend
- how much machine dependent
- Use of more than one IR

Types of Intermediate Languages

- *High Level Representations* (e.g., syntax trees):
  - closer to the source language
  - easy to generate from an input program
  - code optimizations may not be straightforward.

- *Low Level Representations* (e.g., 3-address code, RTL):
  - closer to the target machine;
  - easier for optimizations, final code generation;

Syntax Trees

A *syntax tree* shows the structure of a program by abstracting away irrelevant details from a parse tree.

- Each node represents a computation to be performed;
- The children of the node represents what that computation is performed on.

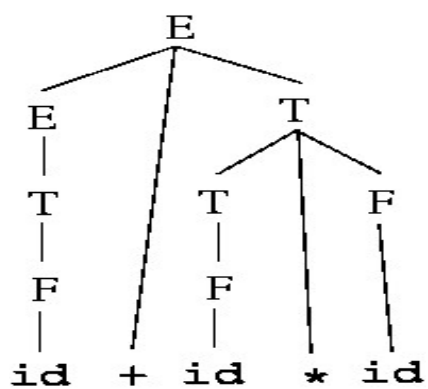
Syntax trees decouple parsing from subsequent processing.

Grammar :

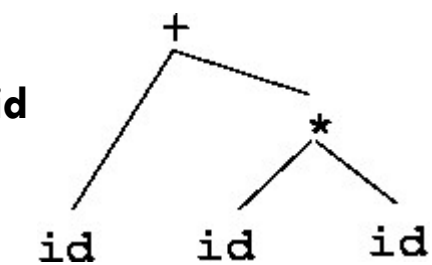
$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \textbf{id}$

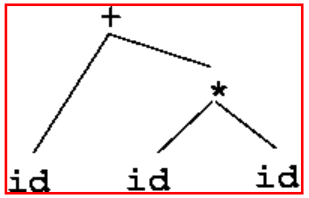


Input: **id + id * id**

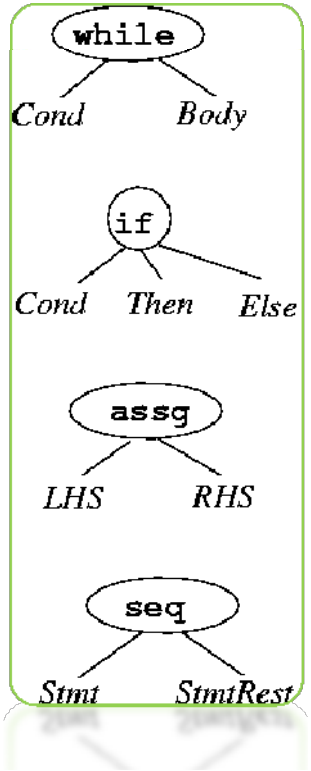


Syntax Trees: Structure

- Expressions:
  - leaves: identifiers or constants;
  - internal nodes are labeled with operators;
  - the children of a node are its operands.

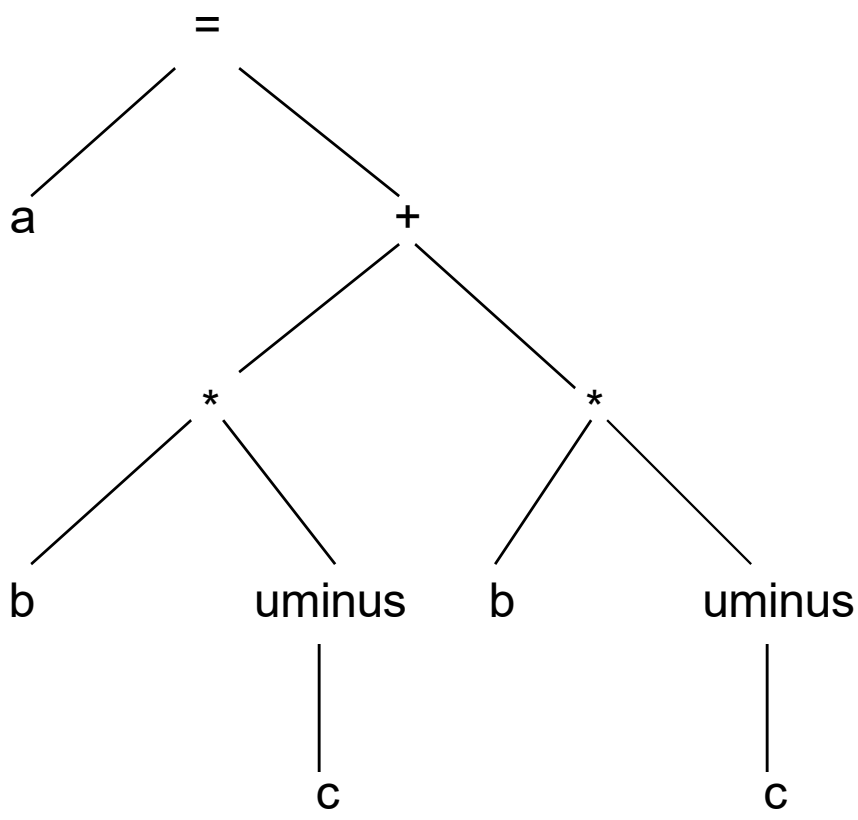


- Statements:
  - a node's label indicates what kind of statement it is;
  - the children correspond to the components of the statement.

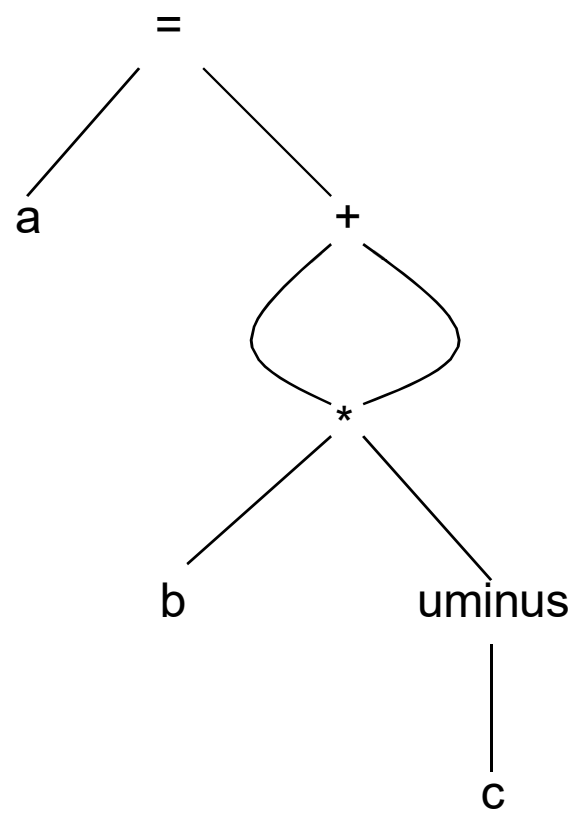


a := b * -c + b * -c

Abstract syntax tree



Directed Acyclic Graph



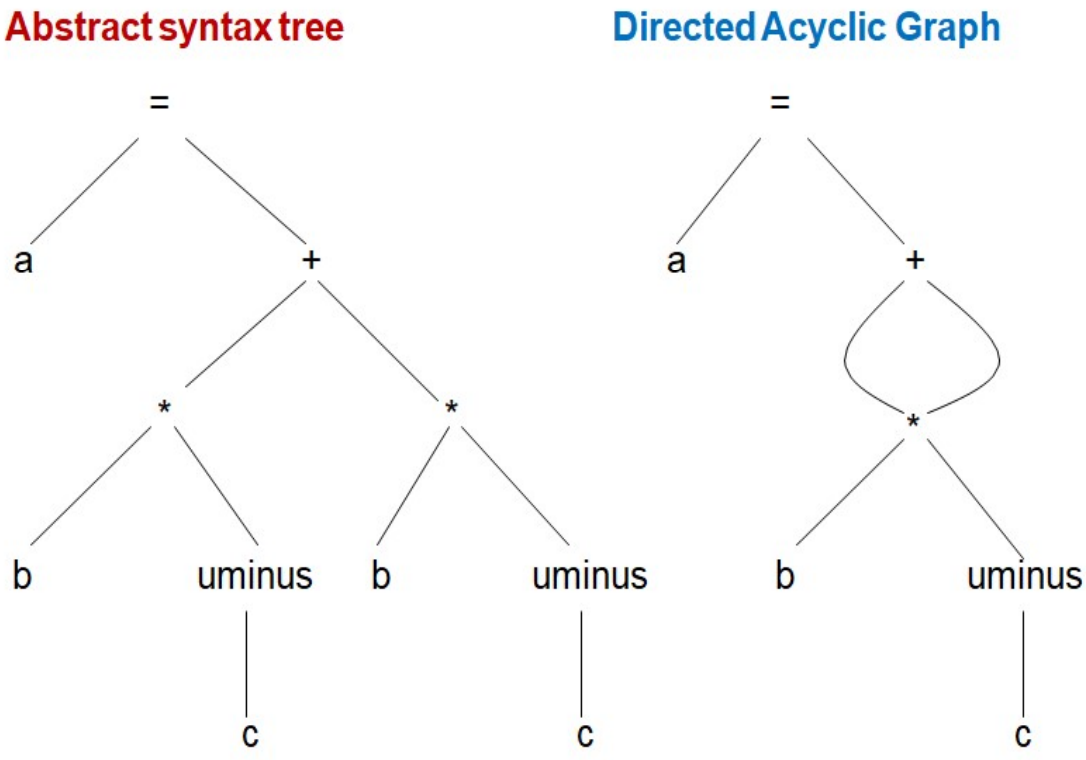
Three Address Code

- Low-level IR
- instructions are of the form '**x = y** ***op*** **z**,' where **x, y, z** are variables, constants, or "temporaries".

- At most one operator allowed on RHS, so no 'built-up" expressions.

  Instead, expressions are computed using temporaries (compiler-generated variables).

a := b * -c + b * -c



3 Address Code:

t1 = -c

t2 = b * t1

t3 = -c

t4 = b * t3

t5 = t2 + t4

a = t5

Optimized:

t1 = -c

t2 = b * t1

t3 = t2 + t2

a = t3

Exercise:

- Draw a syntax tree, DAG and Three address code for the expression:

  $$a + a * (b - c) + (b - c) * d$$

Types of Intermediate Instruction Set

- *Assignment*:
  - x = y op z (op binary)
  - x = op y (op unary);
  - x = y (Copy)
- *Jumps*:
  - if (x op y) goto L (L a label);
  - goto L (**Unconditional Goto)**
- *Indexed Assignments*:
  - x = y[z]
  - y[z] = x
- *Procedure call/return*:
  - param x, k (x is the kth param)
  - call p
  - Return
- *Pointer & Address*:
  - x = &y
  - x = *y
  - *y = x
- *Miscellaneous*
  - label L

Three Address Code: Example

- *Source*:

```
if ( x + y*z > x*y + z)
    a = 0;
```

- *Three Address Code*:

```
tmp1 = y*z
tmp2 = x+tmp1          // x + y*z
tmp3 = x*y
tmp4 = tmp3+z          // x*y + z
if (tmp2 <= tmp4) goto L
a = 0
L:
```

Example

```
do{
i = i+1;
}while (a[i] < v);
```

```
L:      t1 = i + 1
        i = t1
        t2 = i * 8
        t3 = a[t2]
        if t3 < v goto L
```

Symbolic labels

```
100:    t1 = i + 1
101:    i = t1
102:    t2 = i * 8
103:    t3 = a[t2]
104:    if t3 < v goto 100
```

Position numbers

Data structures for three address codes

- Quadruples
  - Has four fields: op, arg1, arg2 and result
- Triples
  - Temporaries are not used and instead references to instructions are made
- Indirect triples
  - In addition to triples we use a list of pointers to triples

Quadruples

- Quadruples
  - Has four fields: op, arg1, arg2 and result

Expression:
a := b * -c + b * -c

3 – addr code:
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

op	arg1	arg2	result
minus	c		t1
*	b	t1	t2
minus	c		t3
*	b	t3	t4
+	t2	t4	t5
=	t5		a

Triples

- Triples
  - Temporaries are not used and instead references to instructions are made

Expression:
a := b * -c + b * -c

3 – addr code:
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Indirect triples

- Indirect triples
  - In addition to triples we use a list of pointers to triples

Expression:
a := b * -c + b * -c

3 – addr code:
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

	op
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Implementation of 3 – address code

Expression:

a := b * -c + b * -c

3 – addr code:

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

Quadruples

op	arg1	arg2	result
minus	c		t1
*	b	t1	t2
minus	c		t3
*	b	t3	t4
+	t2	t4	t5
=	t5		a

Triples

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Indirect Triples

	op
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)